COMPUTER VISION / CS418

Deep Learning for Computer Vision

Neural networks, optimization, and generalization

Mohammad Sabokrou
New Uzbekistan University

Connection to the previous lecture

Previously

We trained models using losses, gradients, and parameter updates.

This lecture

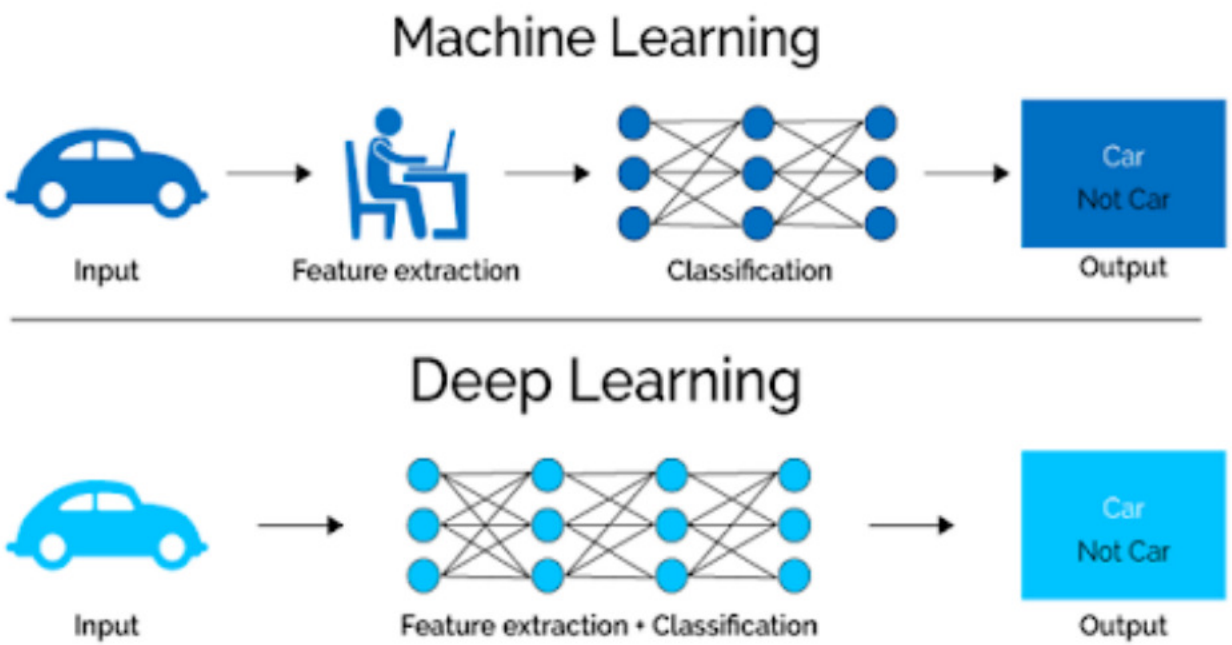
The same learning process now changes a multilayer representation.

The guiding calculation

Input, hidden activations, prediction, loss, gradients, new weights.

The earlier optimizer and loss examples remain useful throughout.

Feature engineering and feature learning



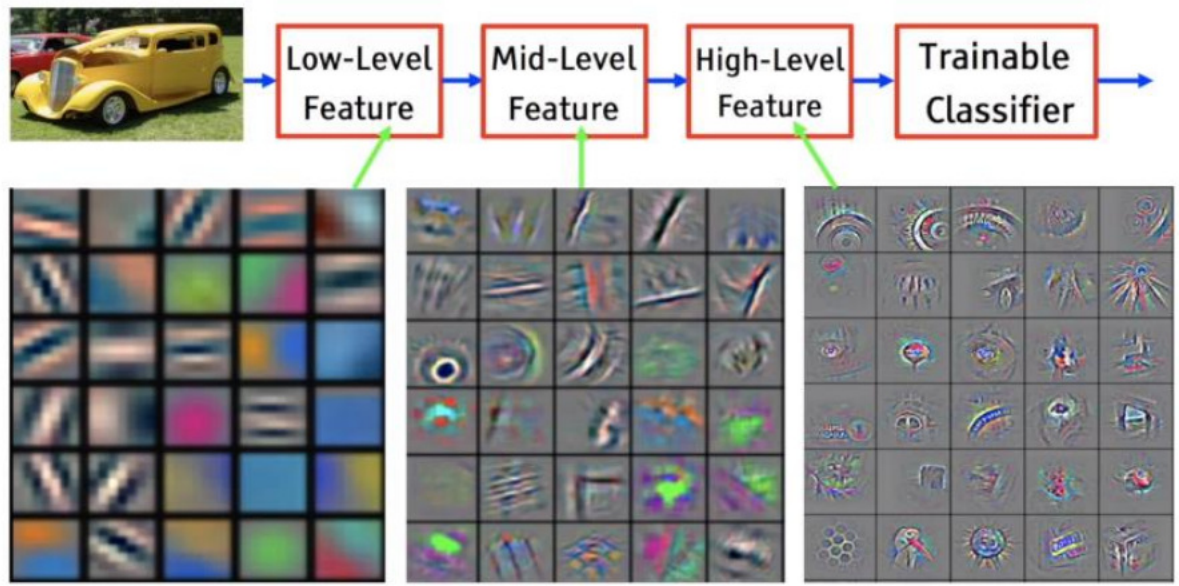
A supplied representation feeds a classifier.

A neural network can learn the representation and the classifier.

For vision, the representation converts image measurements into useful features.

Representations across layers

Different layers can respond to different patterns.



Training determines the features. Their meaning depends on the model and data.

What feature learning changes

Training adjusts the representation that the classifier receives.

$$\mathbf{h} = h_\phi(\mathbf{x}), \qquad \hat{y} = g_\psi(\mathbf{h})$$

Feature extractor

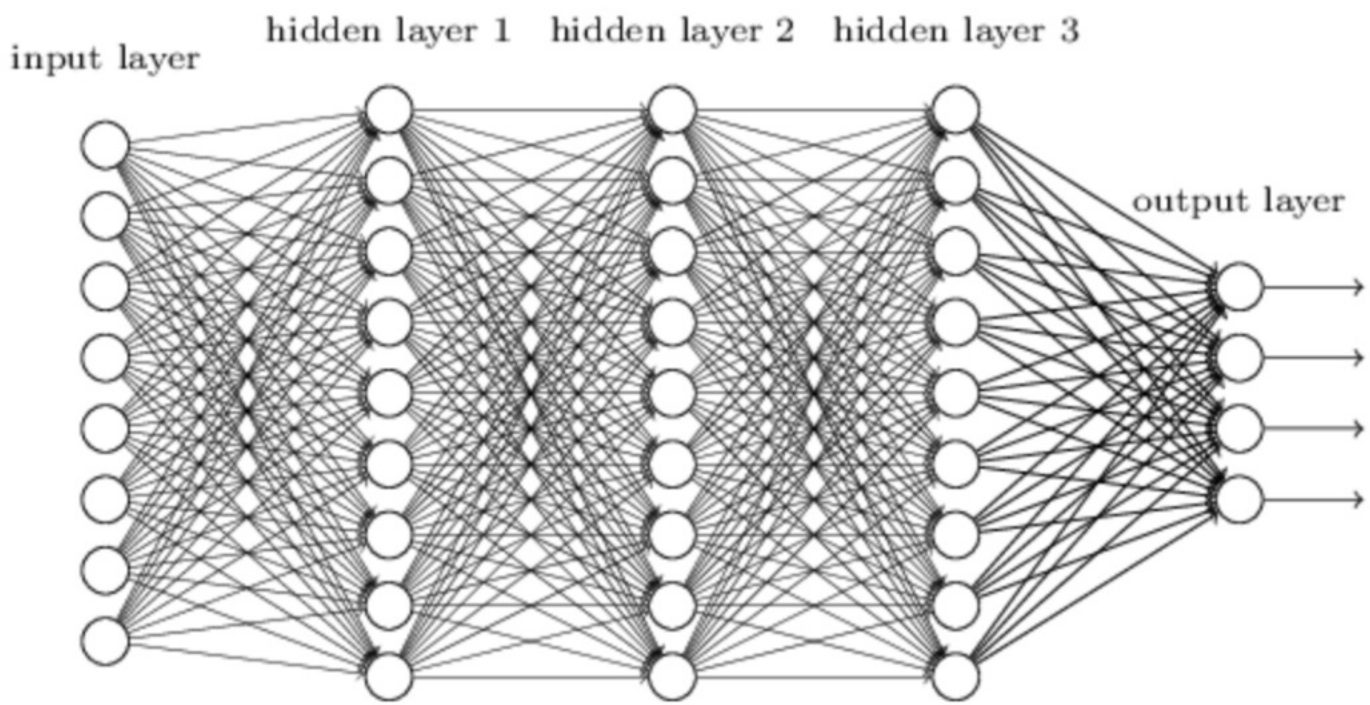
φ: hidden-layer weights and biases.
h: the feature vector they produce.

Classifier

ψ: output-layer weights and biases.
The classifier maps h to a prediction.

The same training loss can update both parameter sets.

The cost of dense layers



178 × 218 × 3
= 116,412 inputs

1,024 hidden units
119,206,912 parameters

10,177 outputs
10,431,425 parameters

Total: 129,638,337 weights and biases in this dense example.

Data, computation, and software

Data

Training examples provide the evidence used to fit a model.

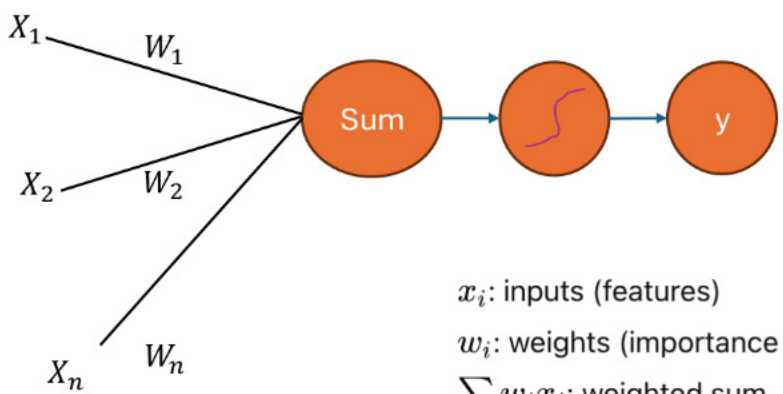
Computation

Accelerated hardware makes large tensor calculations practical.

Software

Automatic differentiation and numerical libraries support training.

A neuron



$$y = f\left(\sum_{i=1}^{n} w_i x_i\right)$$

x_i: inputs (features)

w_i: weights (importance of each input)

$\sum w_i x_i$: weighted sum

$f(\cdot)$: activation function (non-linearity)

y: neuron output

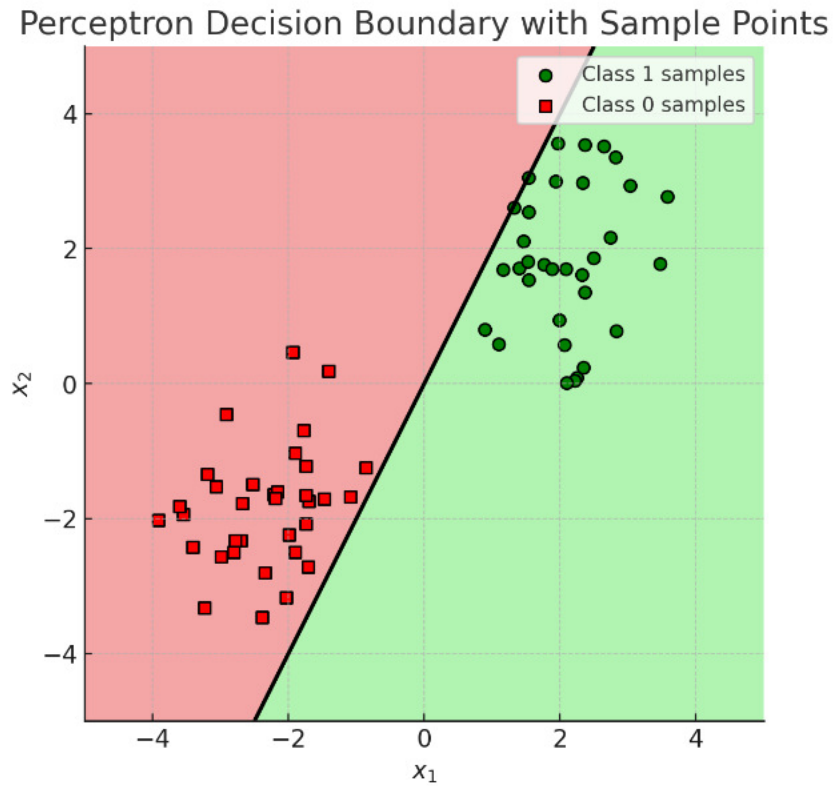
$$a = \sum_{i=1}^{n} w_i x_i$$

$$h = f(a)$$

Weights combine inputs. The activation transforms the sum.

We will add the bias explicitly next.

A linear decision boundary

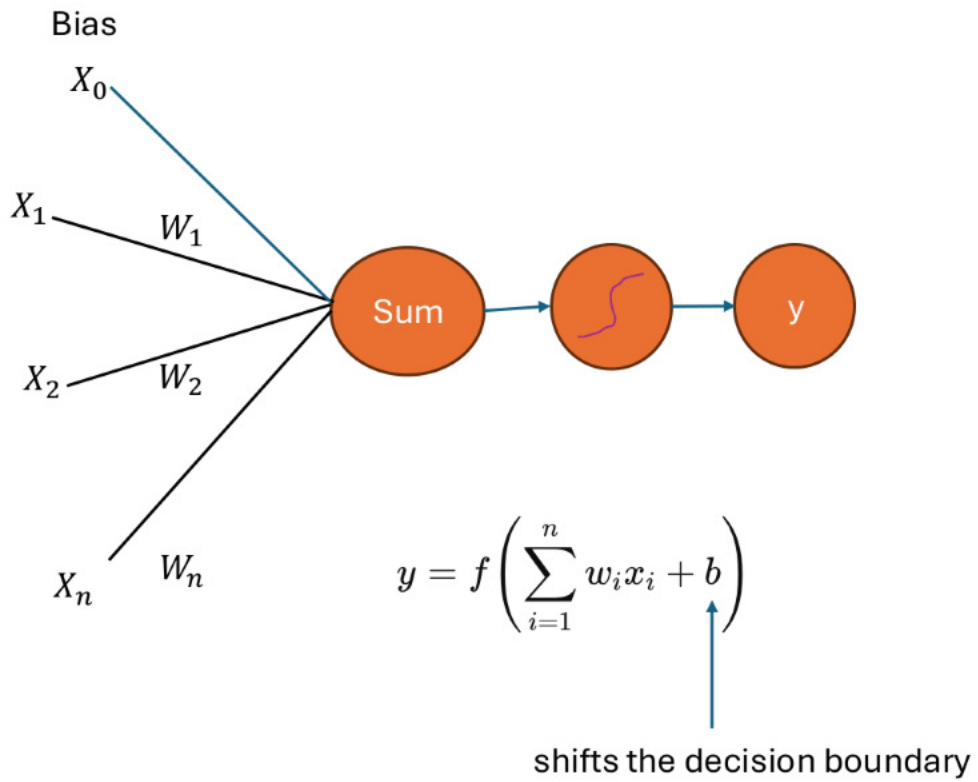


$$a = \mathbf{w}^T\mathbf{x} + b$$

The boundary is where the score equals zero.

A sigmoid changes the score into a probability while preserving this zero-score boundary.

The role of the bias



$$a = \mathbf{w}^T\mathbf{x} + b$$

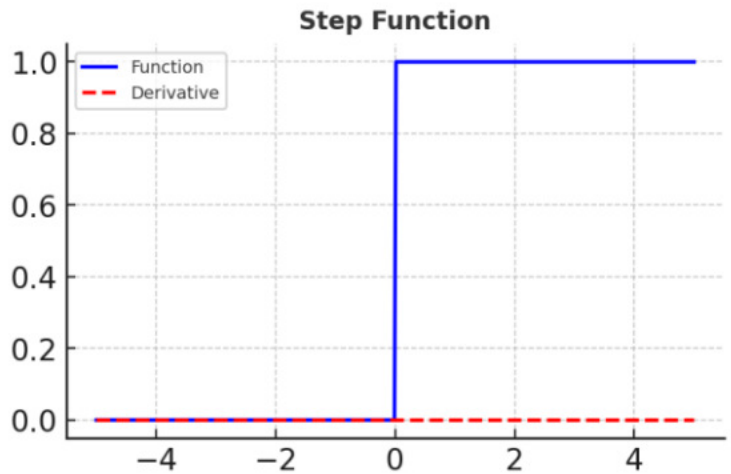
The bias shifts the response.

A constant input of 1 can represent the bias.

Weights and bias are learned parameters.

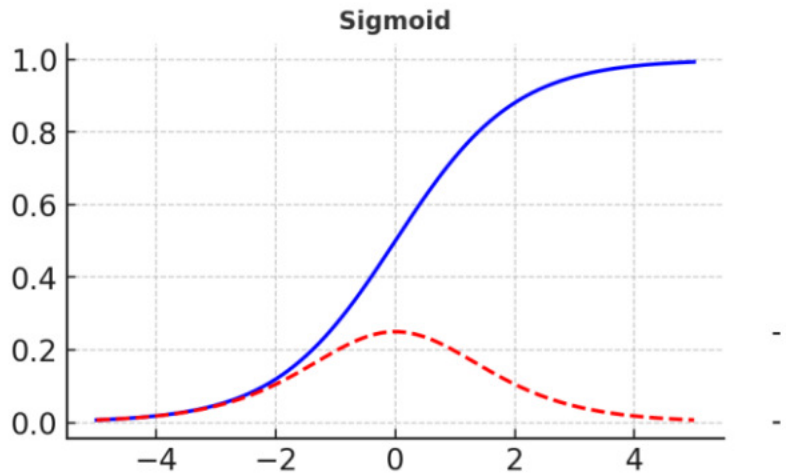
Step and sigmoid activations

Two ways to transform a scalar score



f(z) = 1 if z >= 0, else 0
f'(z) = 0 (undefined at z=0)

Step: a hard threshold



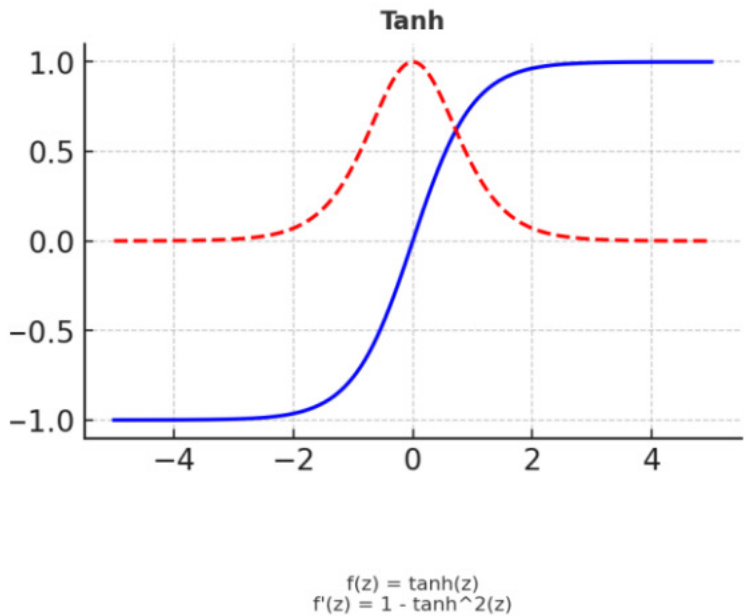
f(z) = 1 / (1 + e^(-z))
f'(z) = f(z) * (1 - f(z))

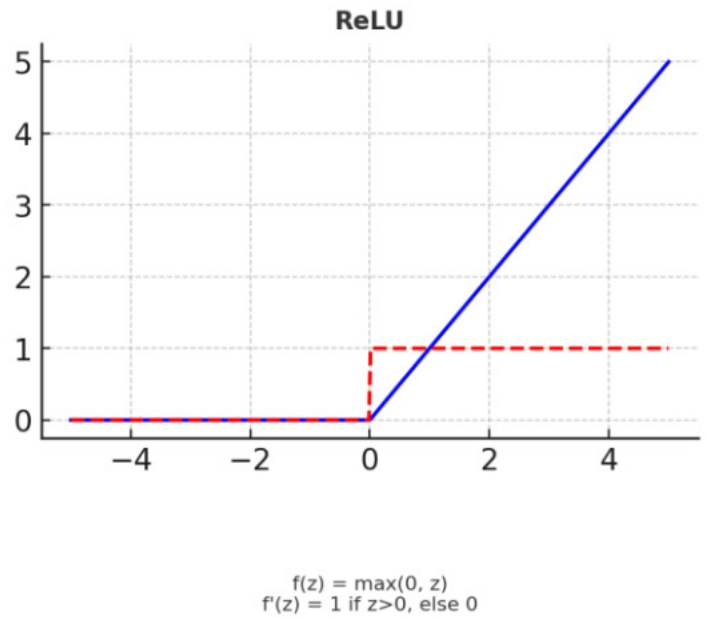
$$\sigma(a) = \frac{1}{1+e^{-a}}$$

A sigmoid output lies between 0 and 1.

Tanh and ReLU activations

Common choices for hidden activations



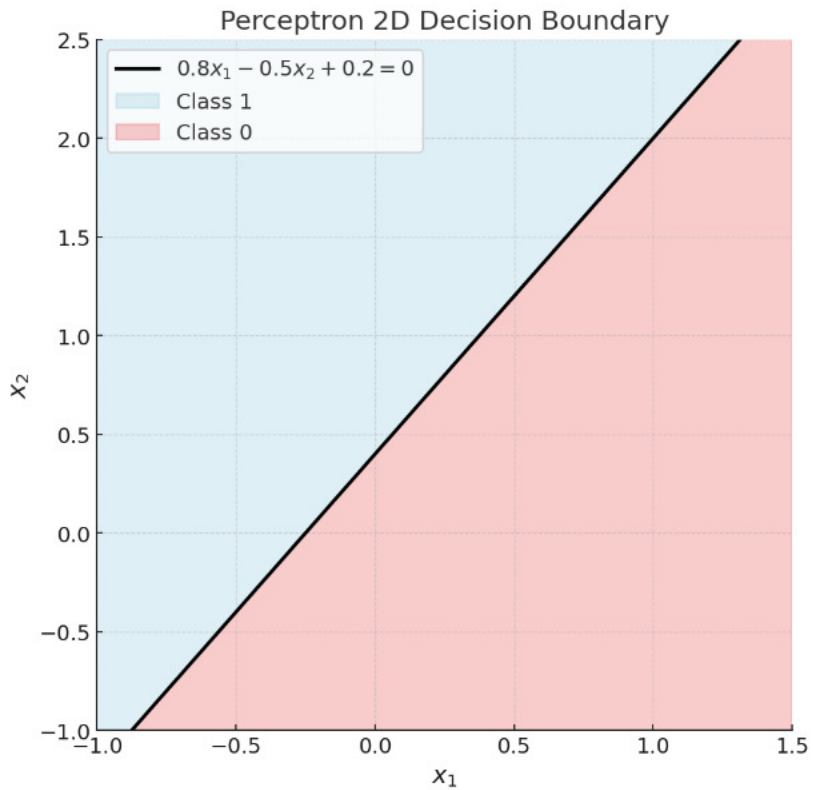


Tanh: output between –1 and 1

$$\mathrm{ReLU}(a) = \max(0, a)$$

Hidden activations are intermediate features, not necessarily probabilities.

Forward pass with two features



$$w_1 = 0.8, \quad w_2 = -0.5$$

$$b = 0.2$$

$$a = 0.8x_1 - 0.5x_2 + 0.2$$

$$p = \sigma(a)$$

$$0.8x_1 - 0.5x_2 + 0.2 = 0$$

A numerical forward pass

For the input x = (0.5, 0.2)

$$a = 0.8(0.5) - 0.5(0.2) + 0.2 = 0.5$$

$$p = \frac{1}{1 + e^{-0.5}} \approx 0.622459$$

At threshold 0.5, predict class 1.

The probability and the thresholded class label are different outputs.

The XOR problem



Class 1: (1,0), (0,1)

Class 0: (0,0), (1,1)

Can one straight boundary separate these classes?

Two boundaries for XOR



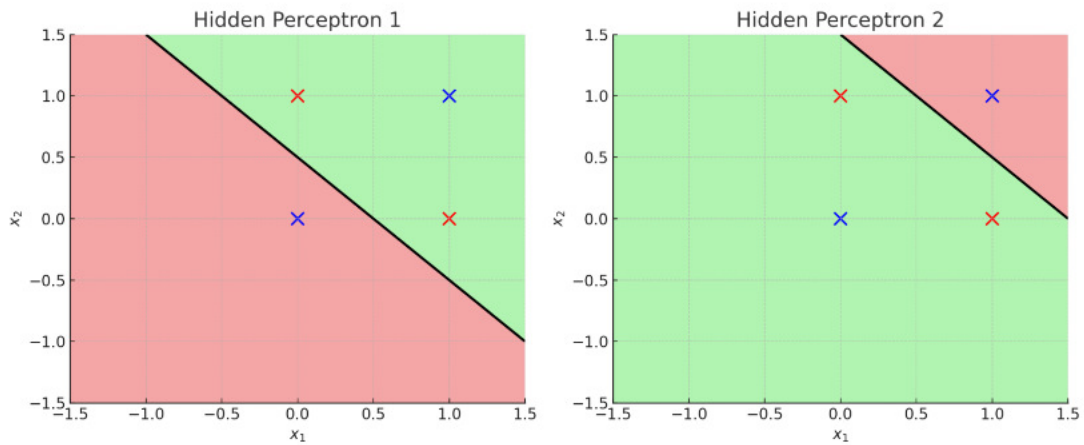
One hidden unit responds on one side of a line.

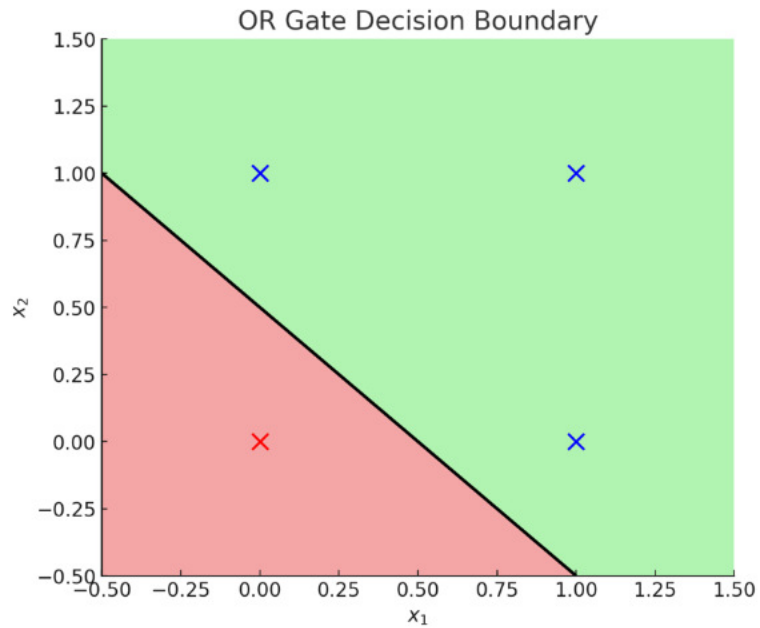
A second hidden unit can define another region.

The next layer combines the hidden responses.

Hidden features separate the classes

Combine two hidden responses to form the output decision.



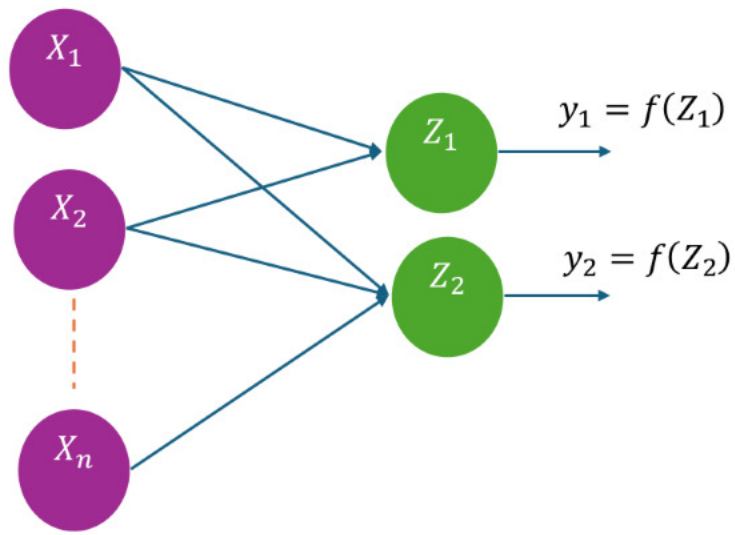


Decisions of the hidden units

Combination at the output

$$h_j = f(\mathbf{w}_j^T \mathbf{x} + b_j), \qquad \text{output} = f(\mathbf{v}^T \mathbf{h} + c)$$

More than one output

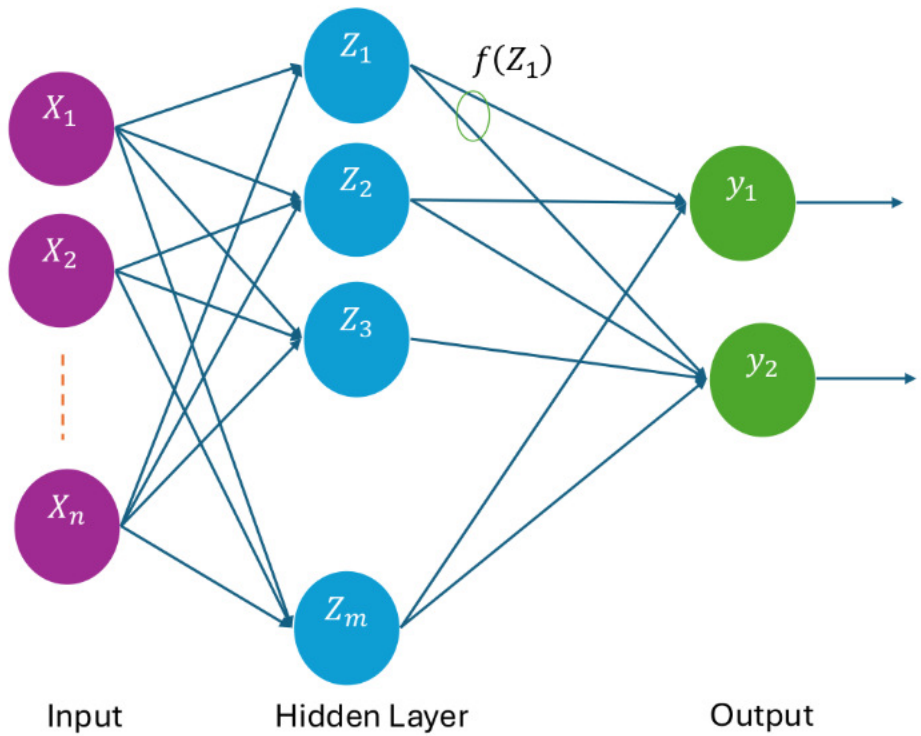


Each output has its own weights.

The outputs share the same input features.

The output activation depends on the task.

A network with a hidden layer



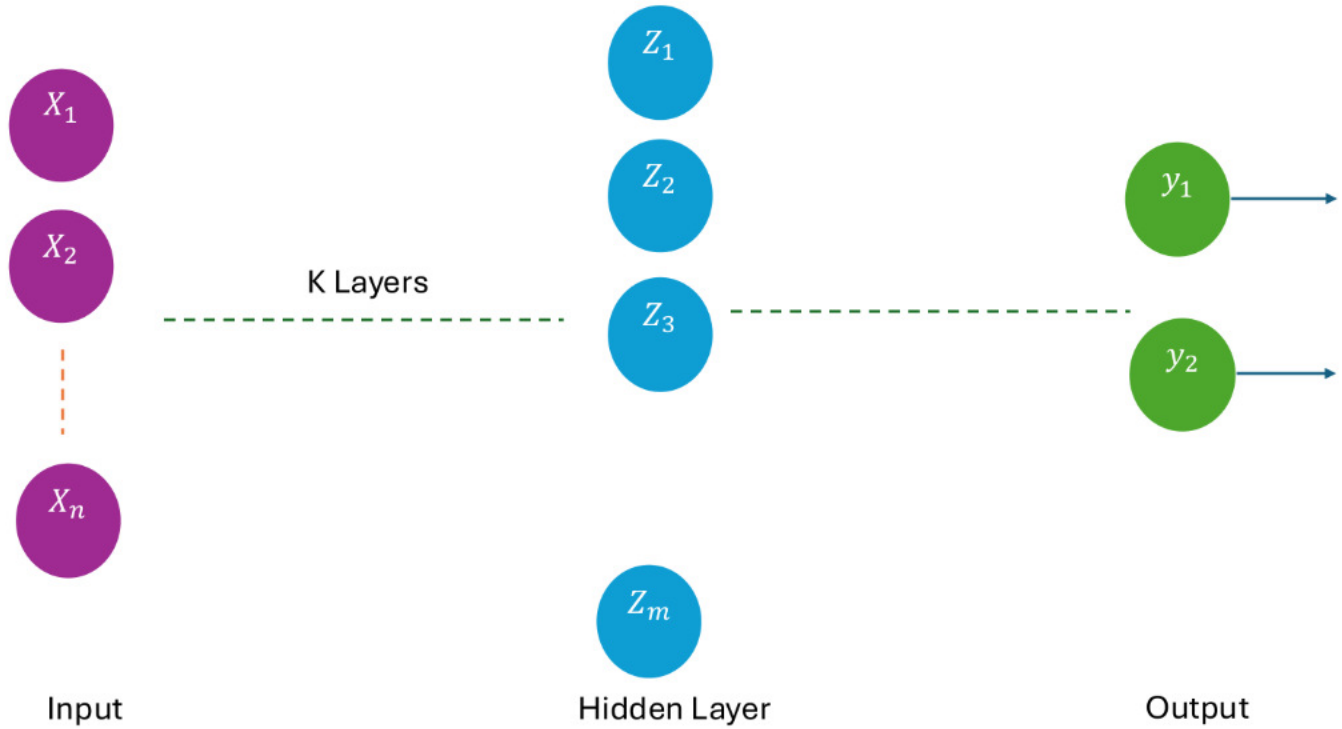
$$\mathbf{h} = f(W_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{z} = W_2\mathbf{h} + \mathbf{b}_2$$

The classifier receives the hidden representation.

The input, hidden activations, and output scores are distinct quantities.

Multiple hidden layers



Each layer transforms the previous representation.

Nonlinear activations allow a richer mapping.

$$\mathbf{h}^{(\ell)} = f(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)})$$

A hidden activation is a feature

Consider two neighboring pixels, with brightness normalized to [0,1].

$$h = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + b)$$

$$w_1 = 1, \quad w_2 = -1, \quad b = 0 \quad \Longrightarrow \quad h = \max(0, x_1 - x_2)$$

Left pixel x_1	Right pixel x_2	Activation h
1	0	1
0	1	0
1	1	0

A larger response indicates stronger left-to-right brightness contrast.

A complete network example

Inputs are height (cm) and weight (kg). Labels come from the toy dataset.

Height	Weight	y	Class
190	85	0	Volleyball
175	100	1	Wrestler
200	92	0	Volleyball
170	110	1	Wrestler

Hidden weights W_1

1.7641	0.4002
0.9787	2.2409
1.8676	−0.9773

$$\mathbf{v} = [0.9501,\ -0.1514,\ -0.1032]^T$$

First input: $x = [190,85]^T$, $y = 0$. No biases in this worked example.

The prediction can be confidently wrong

First example: x = [190,85]^T and the observed label y = 0

$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x}) = [\,369.1960,\ 376.4295,\ 271.7735\,]^T$$

$$z = \mathbf{v}^T\mathbf{h} \approx 265.734668$$

$$p = \sigma(z) \approx 1, \qquad \hat{y} = 1$$

Random initial weights do not solve the classification problem.

Measuring prediction error

Compare observed targets with predicted probabilities.

i	Target	Prediction	Absolute error	Squared error
1	0	0.18	0.18	0.0324
2	0	0.27	0.27	0.0729
3	1	0.81	0.19	0.0361
4	1	0.92	0.08	0.0064
5	0	0.35	0.35	0.1225
6	1	0.74	0.26	0.0676
7	0	0.41	0.41	0.1681
8	1	0.88	0.12	0.0144

$$\mathrm{MAE} = 0.2325, \qquad \mathrm{MSE} = 0.06505$$

Binary and multiclass cross-entropy

The observed label selects the correct-outcome probability.

$$L_{\text{binary}} = -[y \ln p + (1-y)\ln(1-p)]$$

$$L_{\text{multiclass}} = -\sum_{k=1}^{K} y_k \ln p_k$$

$$y_c = 1 \quad \Longrightarrow \quad L = -\ln p_c$$

c is the true class in a one-hot target.

The training objective

Average the loss over N training examples. θ contains all model parameters.

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} L(f(x_i; \theta), y_i)$$

$$\theta^\star = \operatorname{argmin}_\theta J(\theta)$$

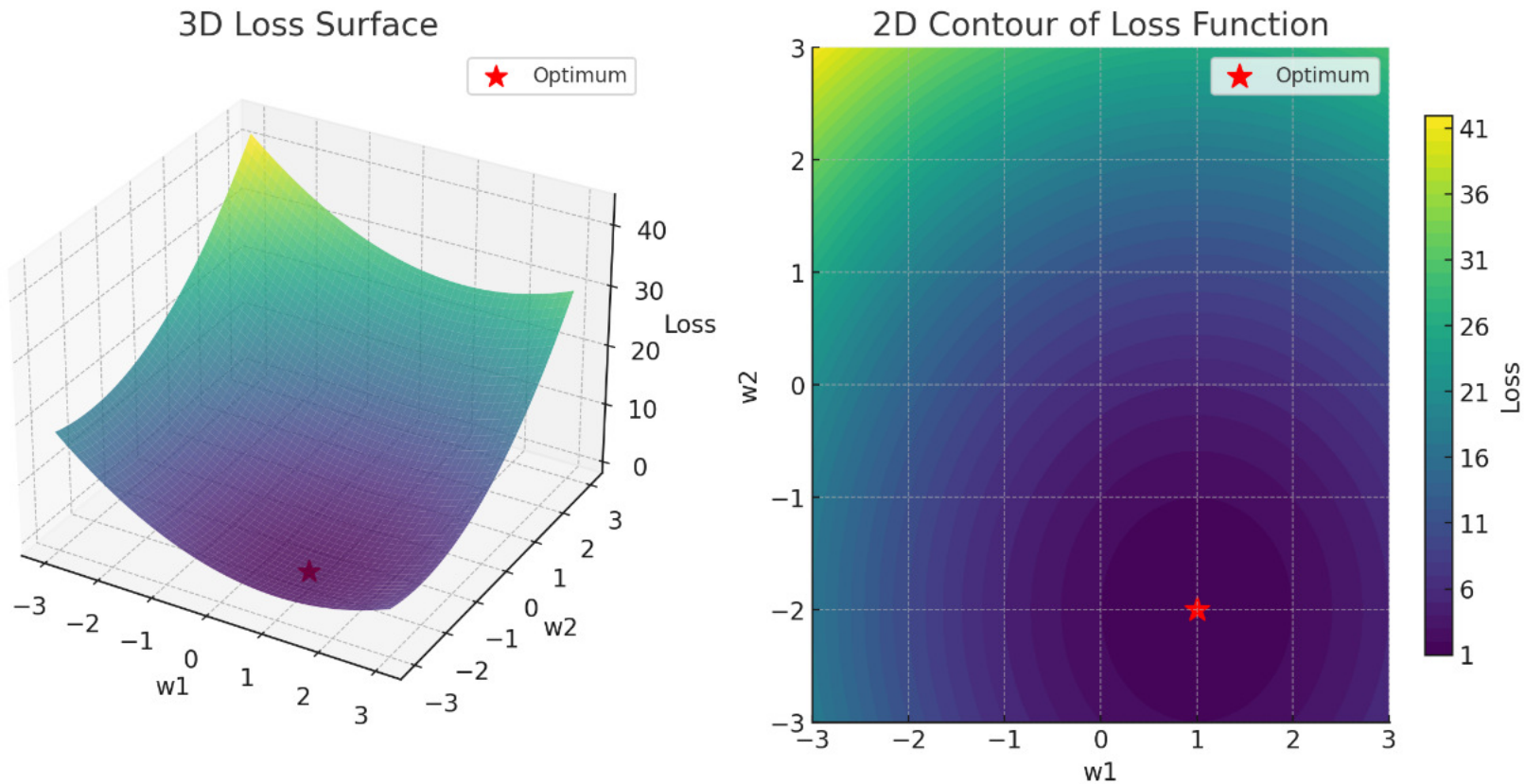
A zero gradient is a stationary point, not automatically a global minimum.

Five-minute break

Next: gradients, optimizers, and generalization.

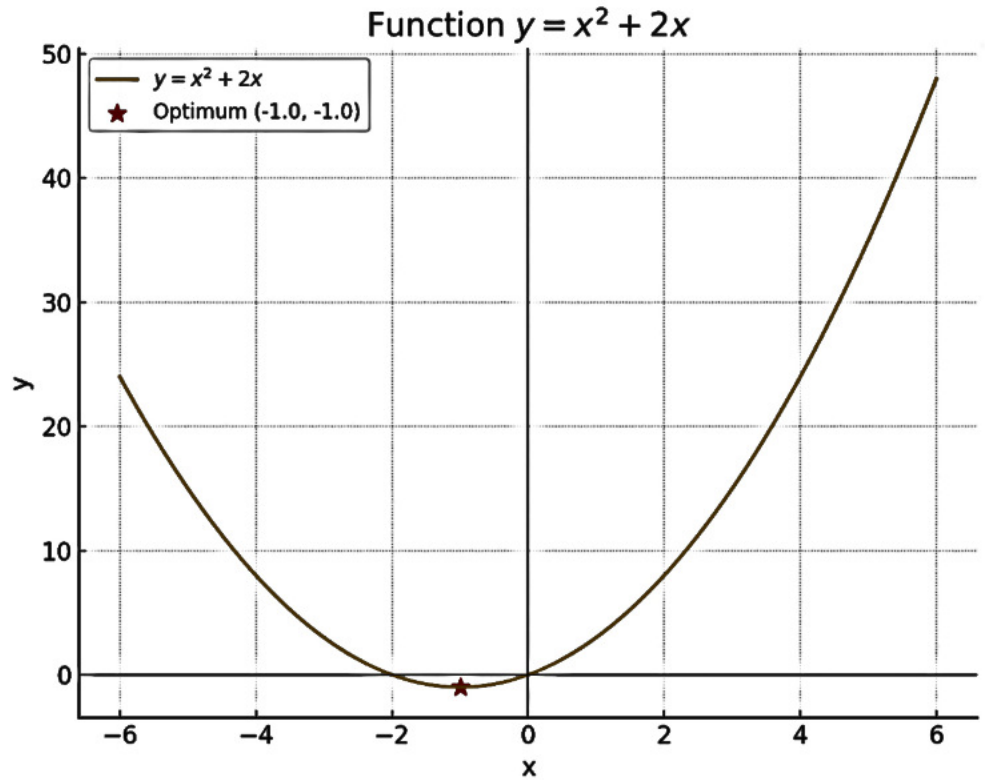
50–55 minutes

A loss surface over parameters



Each point represents one setting of the model parameters.

A derivative gives a local direction



$$J(w) = w^2 + 2w$$

$$\frac{dJ}{dw} = 2w + 2$$

The minimum is w = –1.

$$w = 2,\ \eta = 0.1: \quad g = 6, \quad w_{\text{new}} = 2 - 0.1(6) = 1.4$$

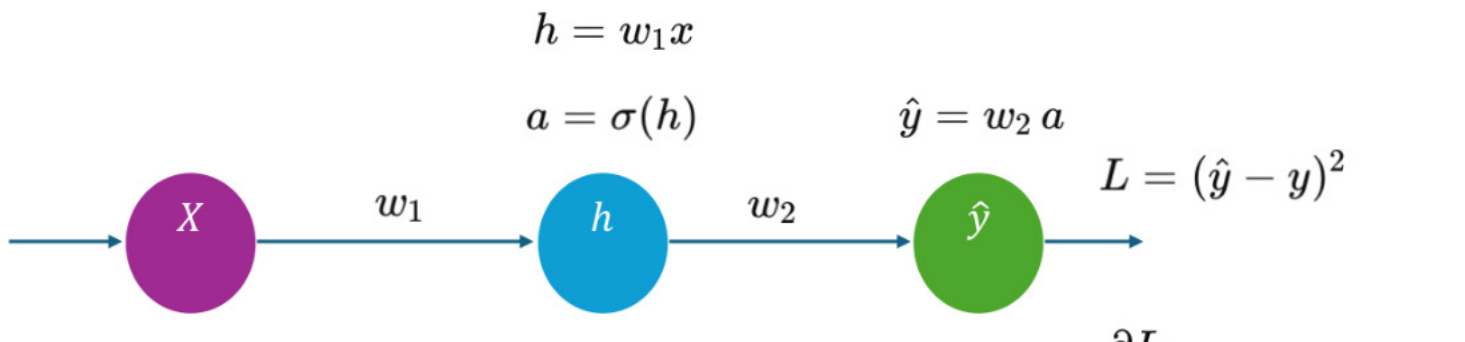
Gradient descent

Repeat the learning cycle

Initialize	Choose starting weights and biases.
Forward and loss	Compute predictions and the training objective.
Differentiate	$g = \nabla_\theta J(\theta)$
Update	$\theta \leftarrow \theta - \eta g$

The same update now applies to every layer's parameters.

Backpropagation through the network



$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = 2(\hat{y} - y) \cdot a$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial h} \cdot \frac{\partial h}{\partial w_1} = \underbrace{2(\hat{y} - y)}_{\text{error}} \cdot \underbrace{w_2}_{\text{next layer weight}} \cdot \underbrace{\sigma'(h)}_{\text{activation slope}} \cdot \underbrace{x}_{\text{input}}$$

Multiply local derivatives along each path from the loss to a parameter.

The loss changes a hidden feature

Use binary cross-entropy with logit h. Update only w_1.

$$\mathbf{x} = (1, 0), \quad w_1 = 1, \quad w_2 = -1, \quad b = 0$$

$$h = 1, \quad p = \sigma(h) \approx 0.7311, \quad y = 0$$

$$\frac{\partial L}{\partial w_1} = (p - y)\frac{\partial h}{\partial w_1} = 0.7311(1) = 0.7311$$

$$w_1^{\text{new}} = 1 - 0.1(0.7311) = 0.9269$$

$$h^{\text{new}} = 0.9269, \qquad p^{\text{new}} \approx 0.7164$$

Other parameters stay fixed. Learning rate: 0.1.

Full-batch gradient descent

Use every training example for one gradient estimate.

$$g = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta L_i(\theta)$$

$$\theta \leftarrow \theta - \eta g$$

The empirical gradient is exact for this dataset, but each update can be expensive.

Stochastic gradient descent

Select one training example for each update.

$$g = \nabla_\theta L_i(\theta)$$

$$\theta \leftarrow \theta - \eta g$$

Individual updates are cheap and noisy.

Mini-batch gradient descent

Use a small group of m training examples.

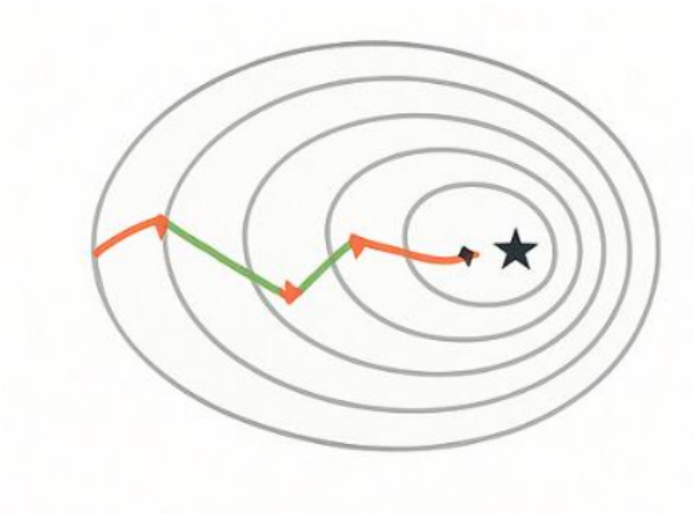
$$g = \frac{1}{m} \sum_{i \in \mathcal{B}} \nabla_\theta L_i(\theta)$$

$$\theta \leftarrow \theta - \eta g$$

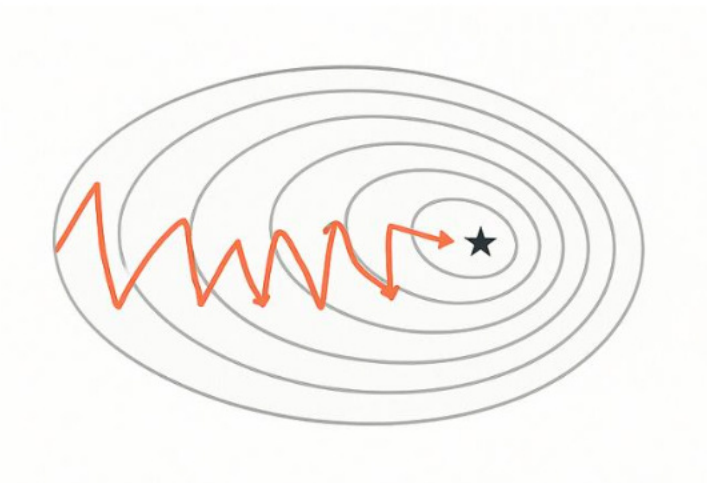
Batch size trades computation per update against gradient variability.

Which method produced each path?

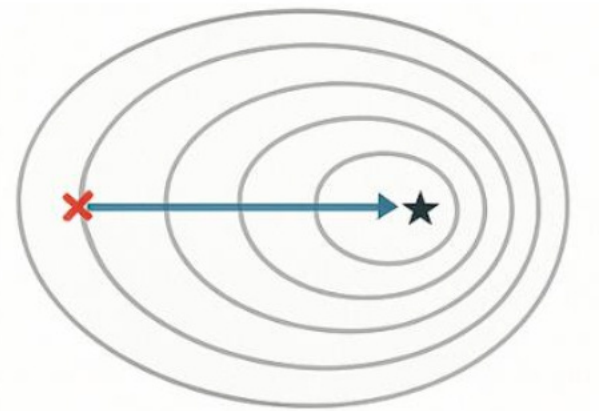
Predict the method before seeing the answer.



A



B

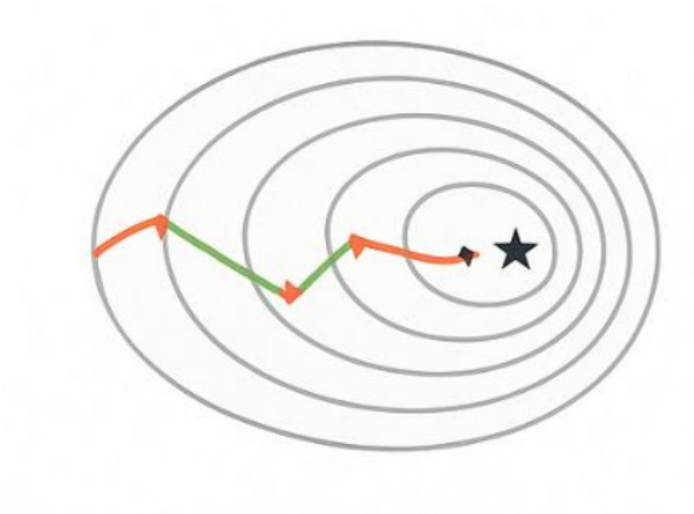


C

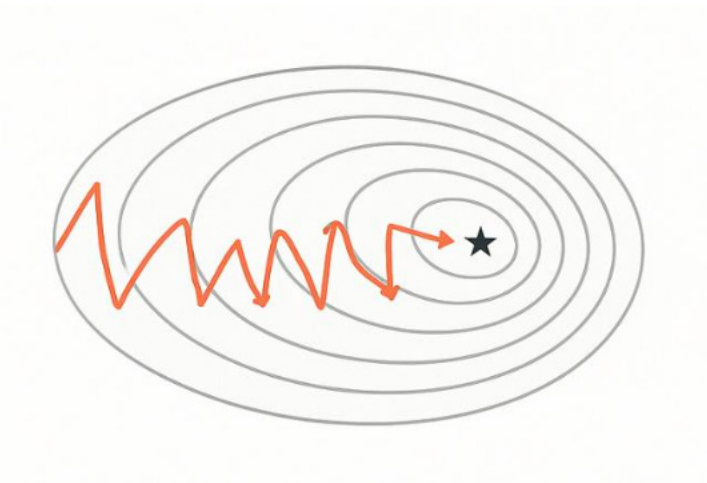
Compare the smoothness of the illustrated trajectories.

The three update strategies

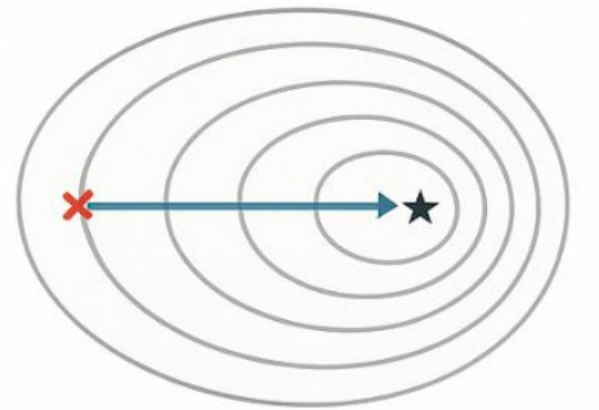
The paths illustrate differences in gradient variability.



Mini-batch



One-example SGD



Full batch

More examples per update usually reduce gradient variability.

The learning rate

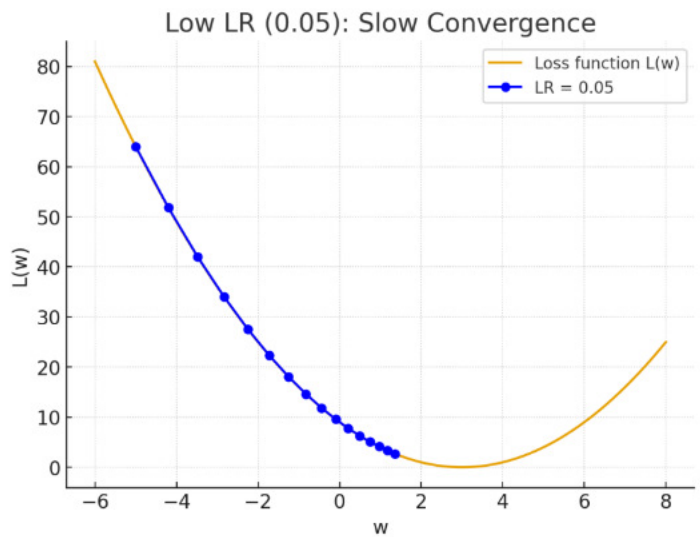
η scales how far the parameters move.

$$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta)$$

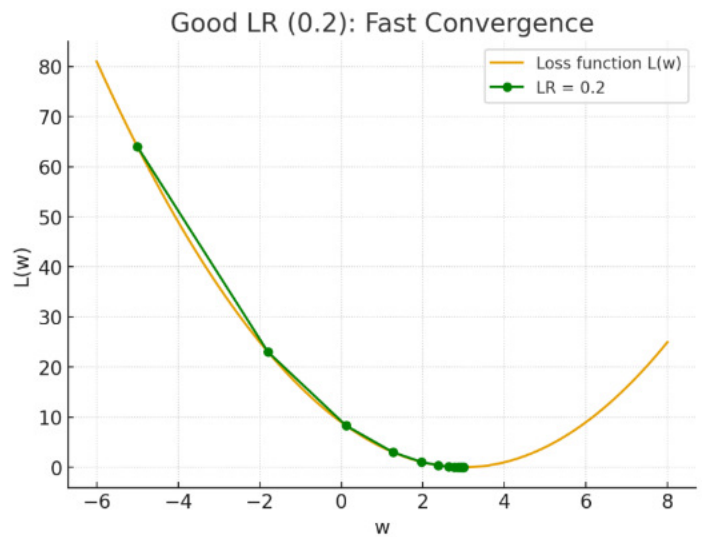
The gradient determines a direction. The learning rate scales the step.

The effect of the learning rate

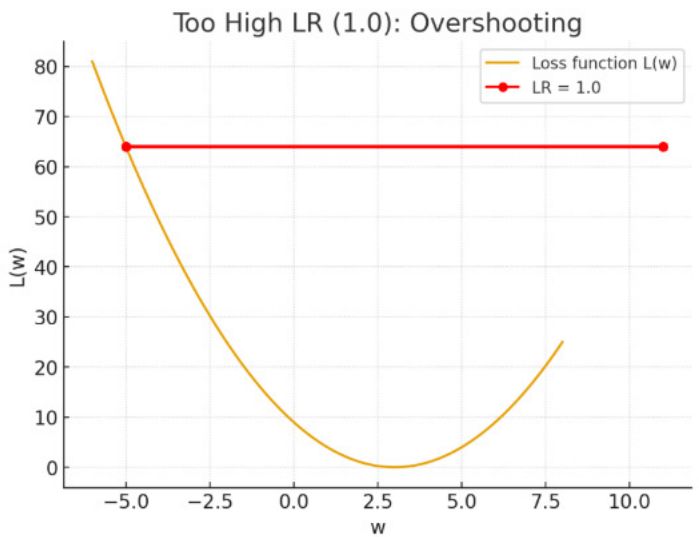
The same starting point with three step sizes



Small step



Moderate step

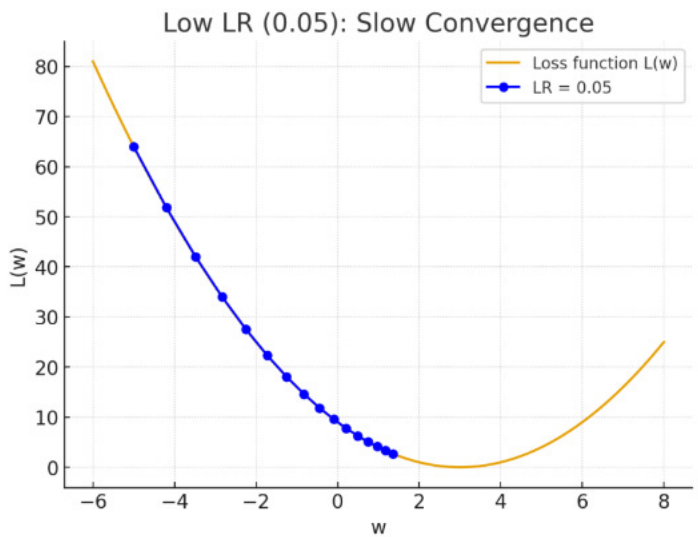


Large step

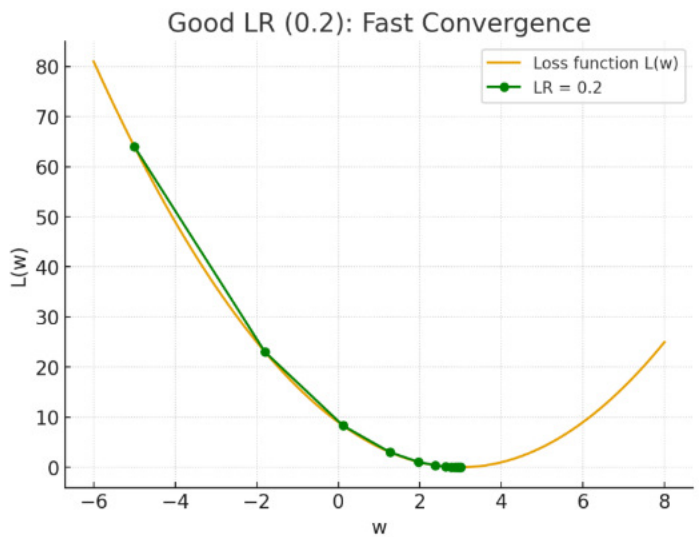
Rates that are too large can oscillate or diverge.

Choosing a learning rate

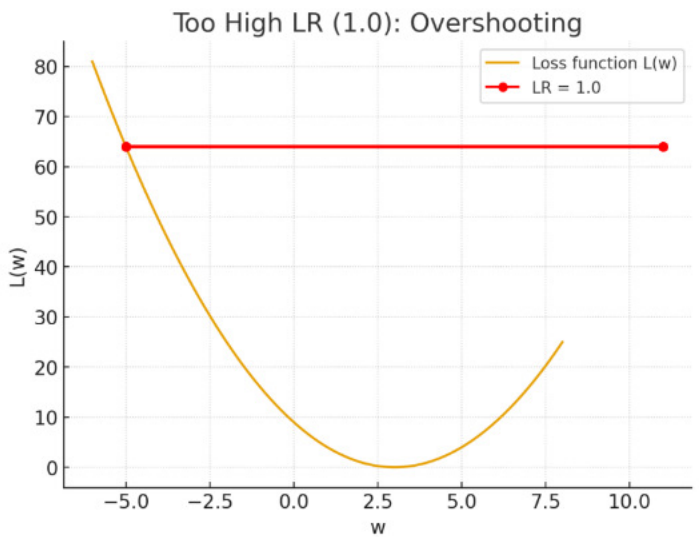
How would you decide which rate to keep?



Slow progress



Useful progress

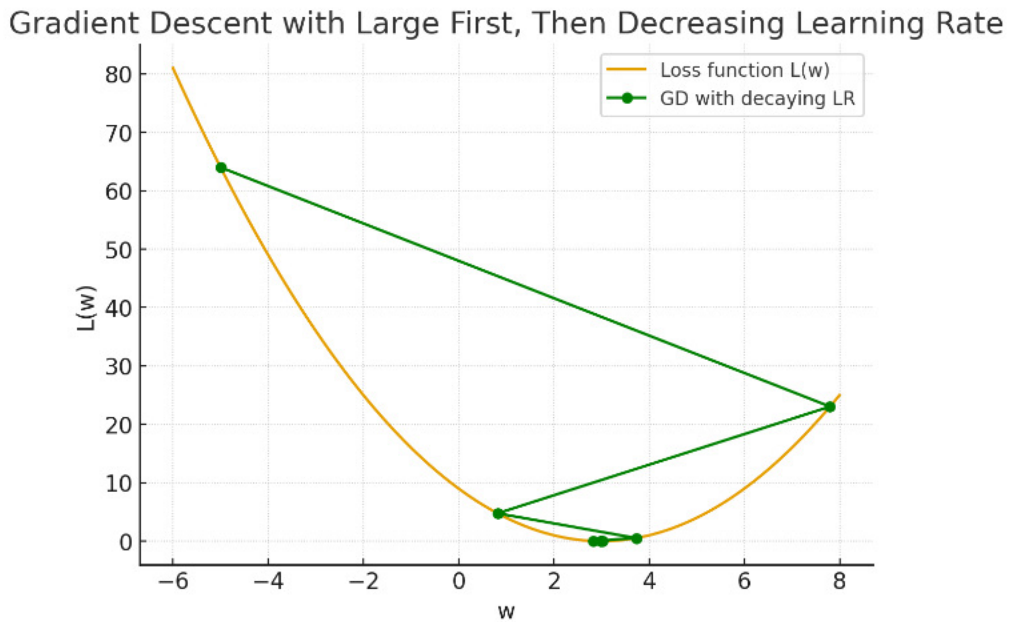


Overshooting

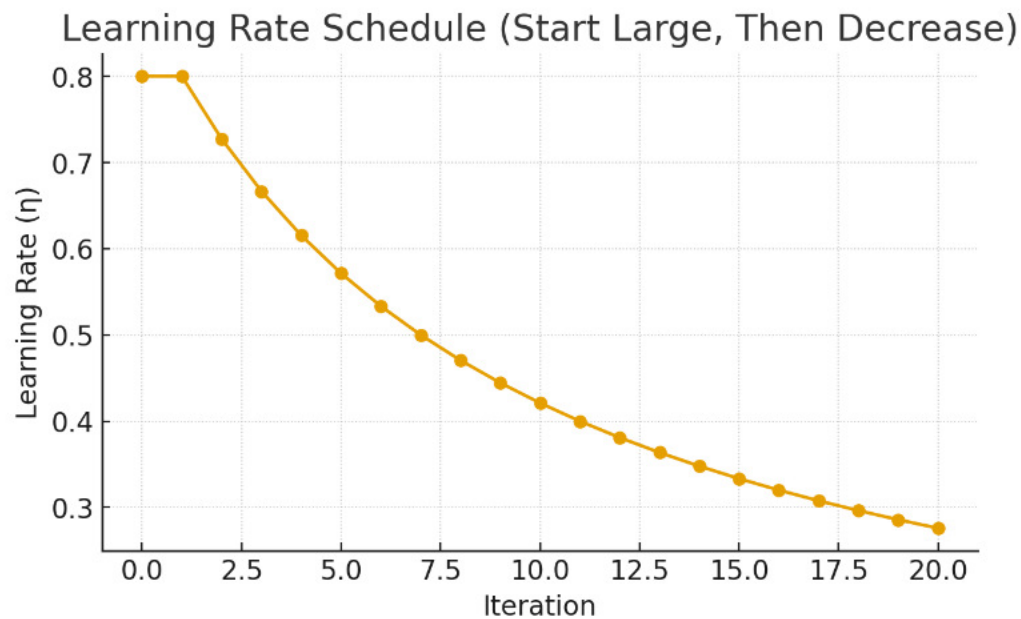
Compare learning behavior, then choose settings using validation data.

Learning-rate schedules

A schedule changes the learning rate during training.



Updates on a loss curve



Learning rate over iterations

Start from a suitable rate, then reduce it as training progresses.

Gradient descent with momentum

Momentum keeps part of the previous accumulated gradient.

$$v_t = \beta v_{t-1} + g_t$$

v_t: new accumulated gradient

t: update number

β

Fraction of old memory kept.
For β = 0.9, keep 90%.

v_{t-1}

Previous accumulated gradient.

$g_t = \nabla_\theta J(\theta_{t-1})$

Current gradient of the loss.

$$\theta_t = \theta_{t-1} - \eta v_t$$

θ: weights and biases
η: learning rate

Memory can smooth the updates and can also cause overshoot.

Adam, one parameter at a time

Repeat these steps for every weight and bias.

1. Gradient	g_t measures how the current loss changes with this parameter.
2. Direction memory	m_t averages recent signed gradients.
3. Squared-size memory	v_t averages recent squared gradients.
4. Update	Correct the memories, scale the direction, then change the parameter.

In Adam, v_t stores squared gradients. It has a different meaning from momentum's v_t.

Adam’s two memories

Each parameter has its own pair of moving averages.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

m_t is the direction memory. m_{t-1} is its previous value.
β_1 keeps part of the old memory, for example 0.9.

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

v_t: squared-gradient memory. v_{t-1}: its previous value.
β_2 keeps old memory (e.g. 0.999). g_t^2 is the squared gradient.

Each (1–β) term gives the weight of the new observation.

Adam’s correction for the zero start

Both memories start at zero. Early averages need a correction.

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}$$

A hat marks a corrected estimate.
The denominators compensate for starting the memories at zero.

First update: $g_1 = -6$, $m_1 = -0.6$, $v_1 = 0.036$.

$$\widehat{m}_1 = \frac{-0.6}{1-0.9} = -6, \qquad \widehat{v}_1 = \frac{0.036}{1-0.999} = 36$$

t counts updates. β_1^t and β_2^t mean the coefficients raised to power t.

Adam’s parameter update

Use a separate history-based scale for every parameter.

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

θ_{t-1} and θ_t

The parameter before and after this update.

η: the learning rate

$\hat{m}_t$	Corrected signed-gradient memory.
$\sqrt{\hat{v}_t} + \varepsilon$	Gradient-size scale plus a small positive ε.
$\varepsilon = 10^{-8}$	Example stabilizer

Subtract the scaled direction. A negative direction increases the parameter.

Adam: the complete update

$$m_0 = v_0 = 0, \qquad t = 1, 2, \ldots$$

1. Gradient

$$g_t = \nabla_\theta J(\theta_{t-1})$$

2. Direction memory

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

3. Squared-size memory

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

4. Zero-start correction

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

5. Parameter update

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Apply these operations elementwise to the parameter vectors.

One Adam step with real numbers

$$J(w) = (w-3)^2, \quad w_0 = 0, \quad \eta = 0.1, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$$

Quantity	First iteration
Gradient g_1	-6
Moving averages	$m_1 = -0.6$, $v_1 = 0.036$
Bias-corrected memories	$\hat{m_1} = -6$, $\hat{v_1} = 36$
Weight after the update	$w_1 \approx 0.1$

$$w_1 = 0 - 0.1 \frac{-6}{\sqrt{36} + 10^{-8}} \approx 0.1$$

$$w_2 \approx 0.199897, \qquad w_3 \approx 0.299618$$

Other optimizers for self-study

Further optimization methods to explore

Methods	Methods
SGD	Adam
SGD with momentum	Nadam
Nesterov accelerated gradient	AdaMax
AdaGrad	AMSGrad
RMSProp	L-BFGS
Adadelta	

Compare the information each method stores and how it forms an update.

Learning and generalization

Fitting training data is one part of evaluating a learned model.

Learning settings

Different learning settings use different training signals.

Setting	Training signal
Supervised learning	Input–target pairs
Unsupervised learning	Structure without supplied target labels
Reinforcement learning	Interaction and rewards

The current forward-pass and loss examples use supervised learning.

Training, validation, and test sets

A supervised dataset contains pairs (x_i, y_i).

Split	Purpose
Training	Fit model parameters
Validation	Select settings and checkpoints
Test	Evaluate the final selected model

Keep related versions of the same original example in the same split.

Fitting a regression model

For numerical targets, one example is a linear predictor with MSE.

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

$$J(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

$$(\mathbf{w}^\star, b^\star) = \operatorname{argmin}_{\mathbf{w}, b} J(\mathbf{w}, b)$$

The model and target determine a suitable loss.

How we evaluate learning

Training performance

Checks how well the fitted model matches the training data.

Validation performance

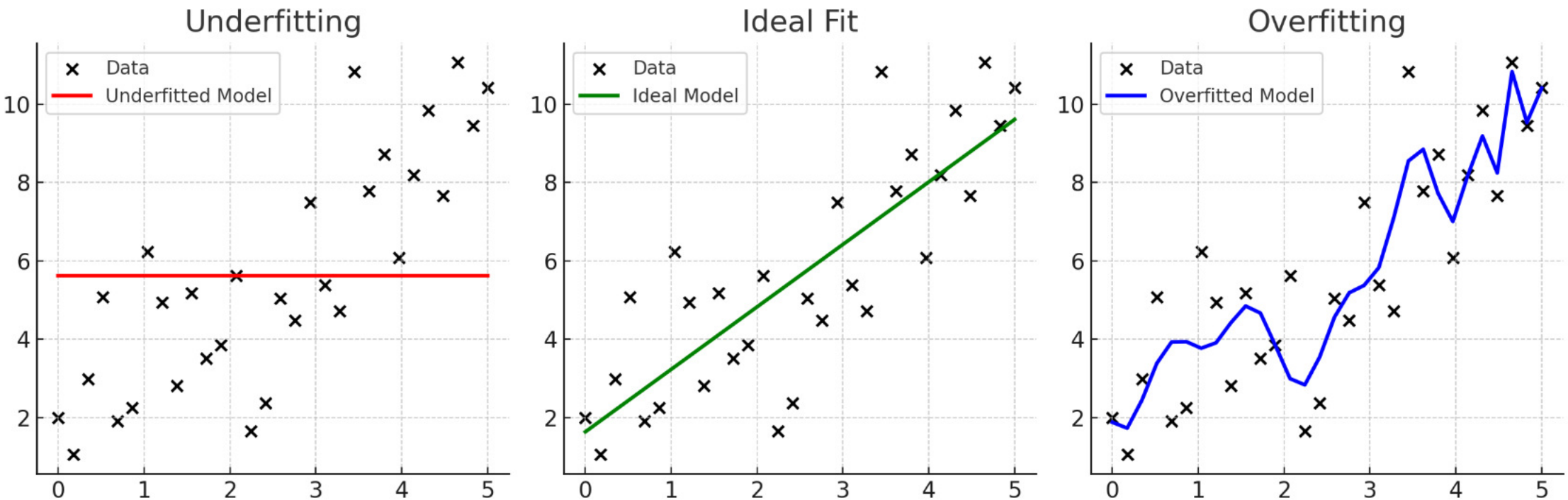
Supports choices about capacity, learning rate, and regularization.

Final test performance

Assesses the selected model on held-out data.

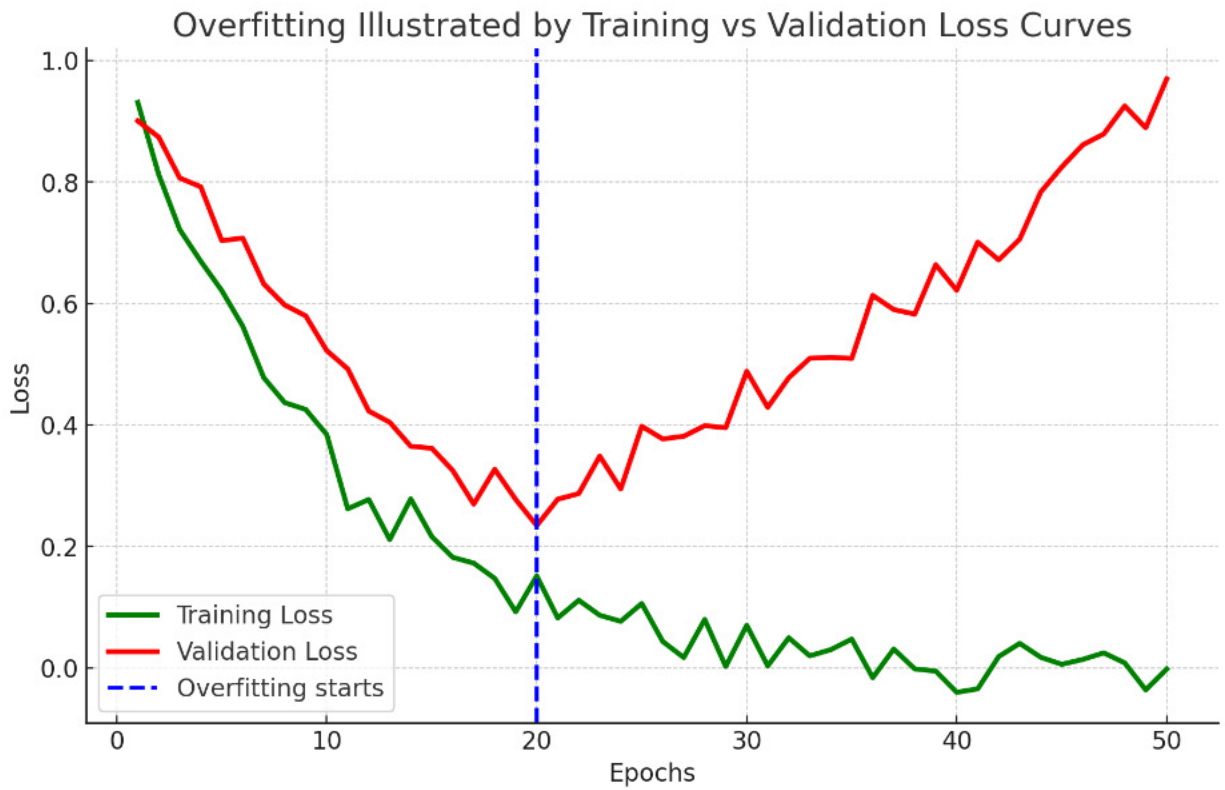
Performance depends on how well the evaluation data represent the intended use.

Underfitting and overfitting



Model flexibility can be too small or too large for the available evidence.

Training and validation loss



Training loss can keep falling.

Validation loss can start rising.

A small gap alone does not prove good generalization: both losses may be high.

Why a model can overfit

Capacity and training

A highly flexible model

Too many training epochs

Insufficient regularization

Available data

Too few examples

Noisy or incorrect labels

Class imbalance

Irrelevant features

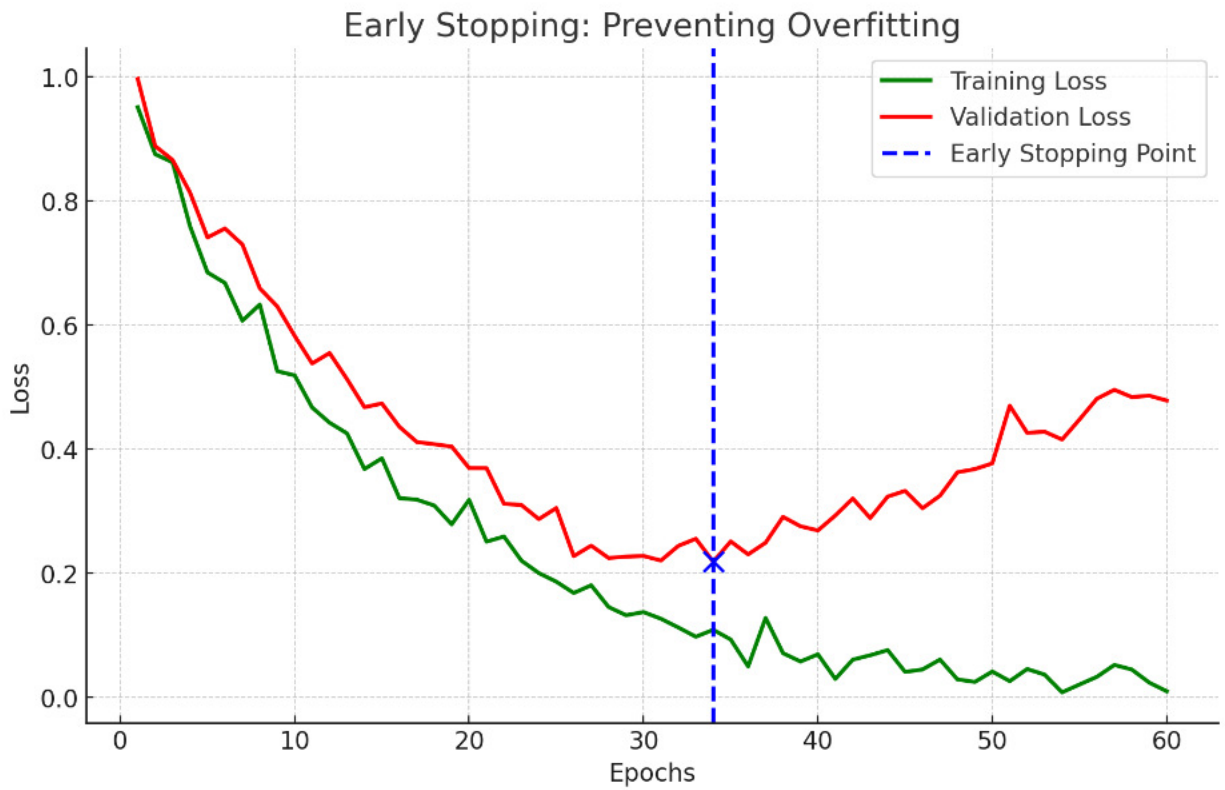
Training choices and regularization

Several training choices can influence held-out performance.

Approach	What it changes
Early stopping	The selected checkpoint
L1, L2, and dropout	Weight or activation constraints
Batch normalization	Intermediate normalization and optimization behavior
Learning-rate schedules	The update scale over time
Noise injection	Training inputs, parameters, or activations

Their effect on held-out performance must be measured.

Early stopping



Monitor validation performance.

Save the best checkpoint.

Stop after a suitable patience period.

The best checkpoint can occur before the stopping time.

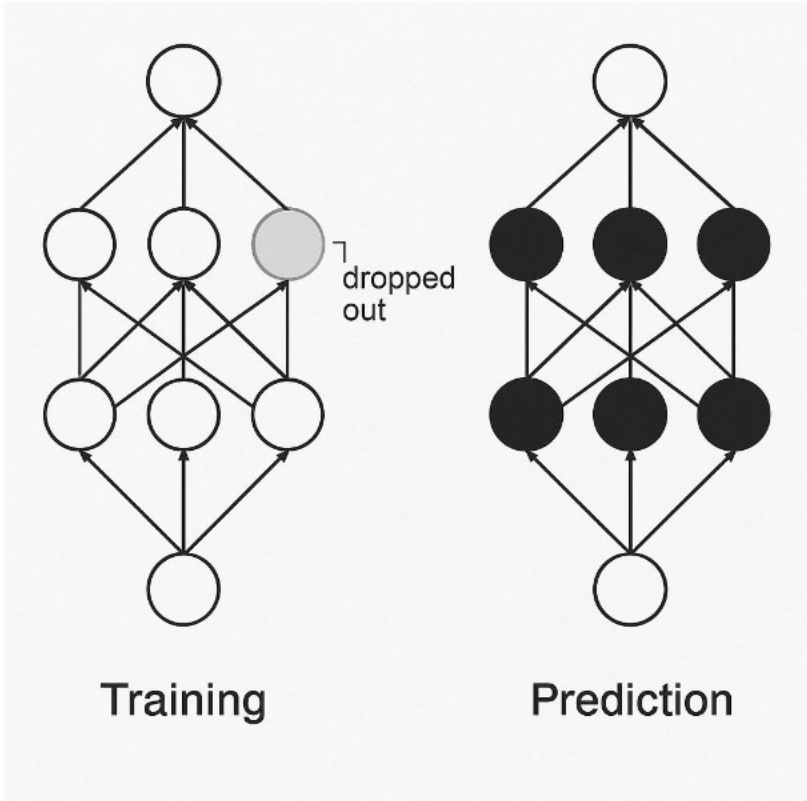
L1, L2, and weight decay

Let L be the data loss and leave the bias unpenalized.

Method	Objective or update
L1	$J = L + \lambda \sum_j \lvert w_j \rvert$
L2	$J = L + (\lambda/2) \sum_j w_j^2$
SGD with weight decay	$w \leftarrow (1 - \eta\lambda)w - \eta\nabla L$

Choose the regularization strength using validation data.

Dropout



Random masks hide activations.

Evaluation uses the complete network.

Retained activations are commonly scaled during training.

The mask is temporary. Dropout does not permanently remove neurons.

Practice after the lecture

1. **Calculate a forward pass through the displayed network.**

2. Trace one weight gradient using the chain rule.

3. Compare training and validation behavior before changing the model.